

MANUEL PROFESSIONNEL ET UNIVERSITAIRE

Consultant en innovation technologique, transformation numérique, cybersécurité, intelligence artificielle, développement logiciel et stratégie digitale.

BASES DE

Contact	Coordonnées
Site web	www.charmantnyungu.com
Email	consultant@charmantnyungu.com
Téléphone	+243 835 137 837
Édition	Formation complète — 2026

Charmant Nyungu

Vision du cours



Ce manuel apprend à concevoir, exploiter et protéger des bases de données professionnelles.

Présentation du manuel

Ce manuel forme à la conception, l'interrogation, l'optimisation, la sécurisation et l'exploitation des bases de données. Il couvre SQL, PostgreSQL, MySQL et MongoDB dans une logique professionnelle : créer des tables, écrire des requêtes, modéliser les relations, utiliser les index, protéger les données et organiser les sauvegardes.

L'auteur, Charmant Nyungu, propose une approche complète destinée aux étudiants, développeurs, administrateurs, data analysts, entrepreneurs et professionnels qui veulent maîtriser les données comme un actif stratégique.

Élément	Détail
Auteur	Charmant Nyungu
Profession	Consultant en innovation technologique, transformation numérique, cybersécurité, intelligence artificielle, développement logiciel et
Site	www.charmantnyungu.com
Email	consultant@charmantnyungu.com
Téléphone	+243 835 137 837

Préface

Une application peut avoir une belle interface et un bon code, mais si sa base de données est mal conçue, elle deviendra lente, fragile et difficile à maintenir. Comprendre les bases de données, c'est apprendre à organiser la mémoire durable d'un système.

Comment utiliser ce livre

- Reproduire chaque requête dans un environnement local.
- Dessiner les relations avant de créer les tables.
- Tester les index avec EXPLAIN plutôt que deviner.
- Documenter les règles de sécurité et les sauvegardes.
- Transformer chaque chapitre en mini projet de portfolio.

Table des matières détaillée

1. Partie 1 : Fondations des bases de données

- Rôle d'une base de données : données, persistance, fiabilité, traçabilité
- Modèle relationnel : table, ligne, colonne, clé
- Modélisation conceptuelle : entité, relation, cardinalité, contrainte

- Normalisation : 1NF, 2NF, 3NF, dépendance

2. Partie 2 : SQL professionnel

- Créer des tables : CREATE TABLE, types, contraintes, clé primaire
- Requêtes SELECT : projection, filtre, tri, limite
- Jointures : INNER JOIN, LEFT JOIN, relation, agrégation
- Transactions : BEGIN, COMMIT, ROLLBACK, isolation
- Vues et procédures : vue, fonction, procédure, réutilisation

3. Partie 3 : PostgreSQL

- Architecture PostgreSQL : schéma, rôle, extension, configuration
- Index PostgreSQL : B-tree, GIN, GiST, partial index
- JSONB et données hybrides : document, requête, index, flexibilité
- Performance et EXPLAIN : plan, coût, scan, optimisation

4. Partie 4 : MySQL

- Architecture MySQL : moteur, InnoDB, schéma, utilisateur
- Requêtes et index MySQL : index, jointure, EXPLAIN, optimisation
- Transactions et verrouillage : ACID, lock, deadlock, isolation
- Administration MySQL : backup, restore, réplication, monitoring

5. Partie 5 : MongoDB et NoSQL

- Logique documentaire : collection, document, champ, embedding
- Requêtes MongoDB : find, filter, projection, update
- Index MongoDB : single field, compound, text, TTL
- Agrégation MongoDB : \$match, \$group, \$project, \$lookup
- Choisir SQL ou NoSQL : structure, volume, flexibilité, consistance

6. Partie 6 : Sécurité, sauvegarde et exploitation

- Sécurité des accès : rôles, permissions, moindre privilège, audit
- Protection des données : chiffrement, secrets, masquage, rétention
- Sauvegarde et restauration : RPO, RTO, dump, test restore
- Monitoring et maintenance : logs, métriques, alertes, vacuum
- Migration et versioning : schema migration, rollback, compatibilité, CI/CD

7. Partie 7 : Projets professionnels

- Base e-commerce : clients, produits, commandes, paiements
- Base hospitalière : patients, consultations, prescriptions, confidentialité
- Base financière : comptes, transactions, audit, fraude
- Base logistique : colis, agences, tracking, preuves
- Base analytique : faits, dimensions, KPI, dashboard

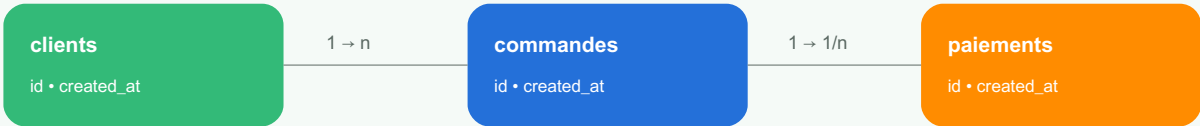
8. Partie 8 : Exercices, certification et annexes

- Banque d'exercices : SQL, relations, index, sécurité
- Examens blancs : requêtes, modélisation, administration, projet
- Glossaire et feuilles de route : termes, références, progression, DBA

Rôle d'une base de données

Ce chapitre traite de rôle d'une base de données avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Schéma relationnel simplifié



Objectifs pédagogiques

- Comprendre le rôle d'une base de données dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
données	Comment maîtriser données dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
persistance	Comment maîtriser persistance dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
fiabilité	Comment maîtriser fiabilité dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
traçabilité	Comment maîtriser traçabilité dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

données

données est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. données aide à rendre ces archives exploitables et sûres.

Cycle de vie des données**persistance**

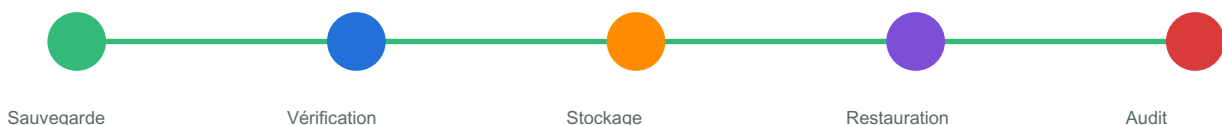
persistance est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. persistance aide à rendre ces archives exploitables et sûres.

fiabilité

fiabilité est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. fiabilité aide à rendre ces archives exploitables et sûres.

Chaîne sauvegarde et restauration**traçabilité**

traçabilité est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. traçabilité aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

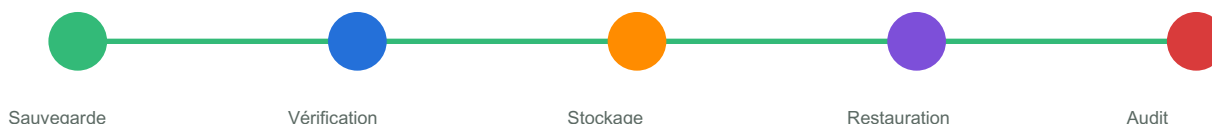
Code — Rôle d'une base de données

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de rôle d'une base de données. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Chaîne sauvegarde et restauration

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à données. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à persistance. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à fiabilité. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à traçabilité. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à données. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Modèle relationnel

Ce chapitre traite de modèle relationnel avec une approche pratique et professionnelle. L’objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Cycle de vie des données



Objectifs pédagogiques

- Comprendre le rôle de modèle relationnel dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l’audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
table	Comment maîtriser table dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
ligne	Comment maîtriser ligne dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
colonne	Comment maîtriser colonne dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
clé	Comment maîtriser clé dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

table

table est une notion essentielle pour construire une base durable. Une bonne base de données n’est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l’historique d’un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l’organisation perd du temps et prend des risques. table aide à rendre ces archives exploitables et sûres.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

ligne

ligne est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. ligne aide à rendre ces archives exploitables et sûres.

colonne

colonne est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. colonne aide à rendre ces archives exploitables et sûres.

Maturité base de données



clé

clé est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. clé aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Modèle relationnel

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de modèle relationnel. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Maturité base de données

SQL



Index



Sécurité



Backup



NoSQL

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser SELECT * dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à table. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à ligne. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à colonne. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à clé. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à table. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

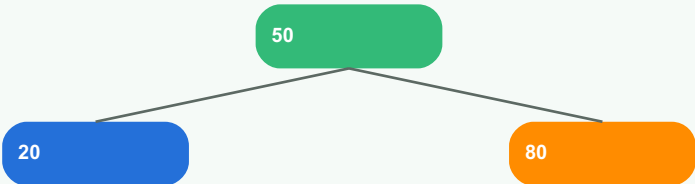
Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Modélisation conceptuelle

Ce chapitre traite de modélisation conceptuelle avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

Objectifs pédagogiques

- Comprendre le rôle de modélisation conceptuelle dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
entité	Comment maîtriser entité dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
relation	Comment maîtriser relation dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
cardinalité	Comment maîtriser cardinalité dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
contrainte	Comment maîtriser contrainte dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

entité

entité est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. entité aide à rendre ces archives exploitables et sûres.

Chaîne sauvegarde et restauration



relation

relation est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

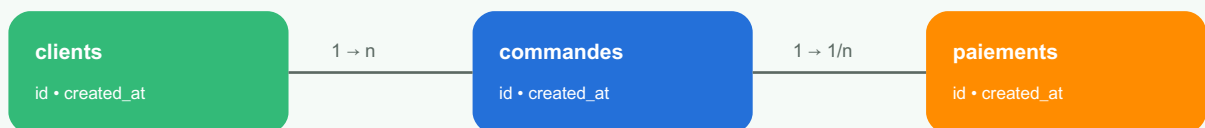
Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. relation aide à rendre ces archives exploitables et sûres.

cardinalité

cardinalité est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. cardinalité aide à rendre ces archives exploitables et sûres.

Schéma relationnel simplifié



contrainte

contrainte est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. contrainte aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

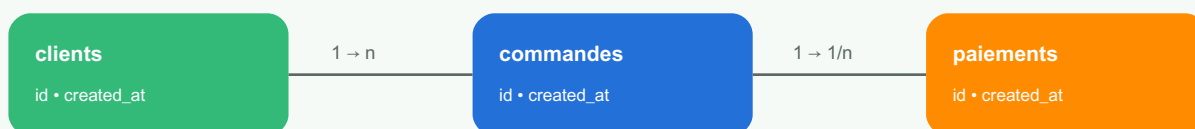
Code — Modélisation conceptuelle

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de modélisation conceptuelle. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Schéma relationnel simplifié

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à entité. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à relation. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à cardinalité. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à contrainte. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à entité. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

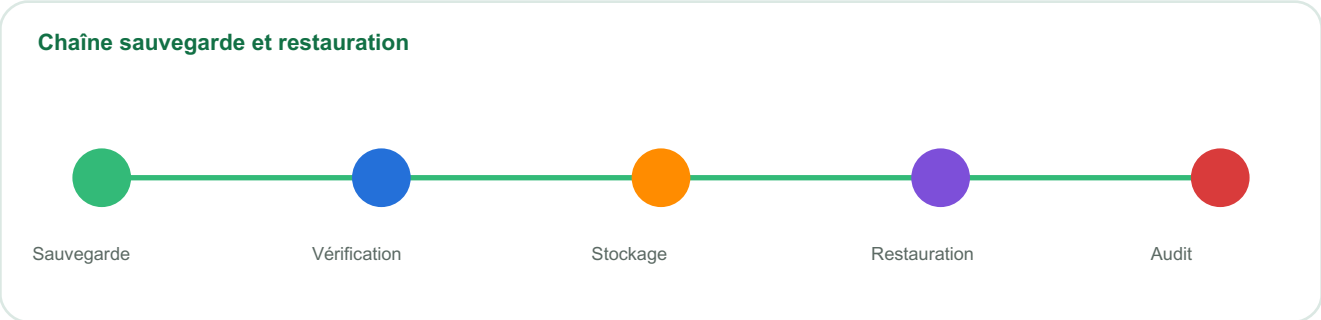
Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Normalisation

Ce chapitre traite de normalisation avec une approche pratique et professionnelle. L’objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.



Objectifs pédagogiques

- Comprendre le rôle de normalisation dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l’audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
1NF	Comment maîtriser 1NF dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
2NF	Comment maîtriser 2NF dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
3NF	Comment maîtriser 3NF dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
dépendance	Comment maîtriser dépendance dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

1NF

1NF est une notion essentielle pour construire une base durable. Une bonne base de données n’est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l’historique d’un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l’organisation perd du temps et prend des risques. 1NF aide à rendre ces archives exploitables et sûres.

Maturité base de données**2NF**

2NF est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. 2NF aide à rendre ces archives exploitables et sûres.

3NF

3NF est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. 3NF aide à rendre ces archives exploitables et sûres.

Cycle de vie des données**dépendance**

dépendance est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. dépendance aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Normalisation

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de normalisation. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Cycle de vie des données

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à 1NF. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à 2NF. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à 3NF. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à dépendance. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à 1NF. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

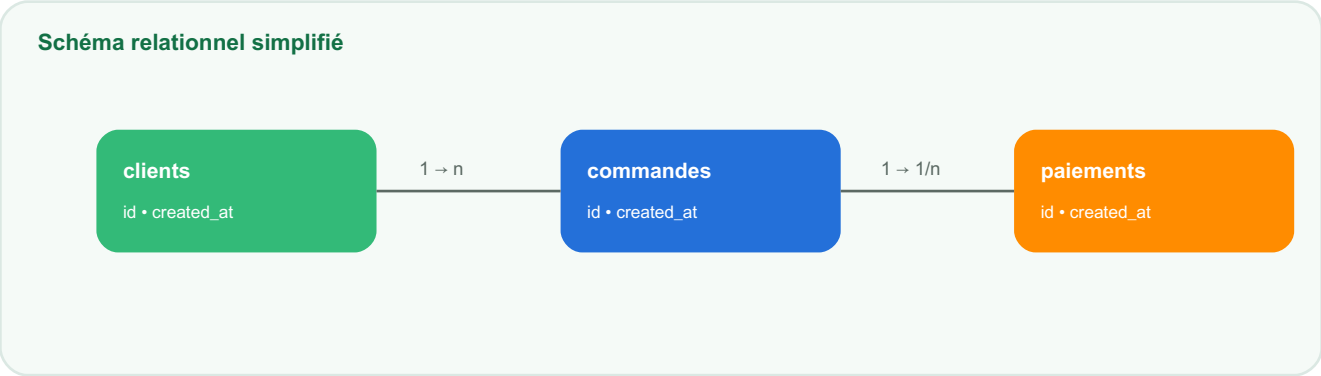
Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Créer des tables

Ce chapitre traite de créer des tables avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.



Objectifs pédagogiques

- Comprendre le rôle de créer des tables dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
CREATE TABLE	Comment maîtriser CREATE TABLE dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
types	Comment maîtriser types dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
contraintes	Comment maîtriser contraintes dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
clé primaire	Comment maîtriser clé primaire dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

CREATE TABLE

CREATE TABLE est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. CREATE TABLE aide à rendre ces archives exploitables et sûres.

Cycle de vie des données**types**

types est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. types aide à rendre ces archives exploitables et sûres.

contraintes

contraintes est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. contraintes aide à rendre ces archives exploitables et sûres.

Chaîne sauvegarde et restauration**clé primaire**

clé primaire est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. clé primaire aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Créer des tables

```
CREATE TABLE clients (
  id SERIAL PRIMARY KEY,
  nom VARCHAR(120) NOT NULL,
  telephone VARCHAR(30) UNIQUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de créer des tables. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Chaîne sauvegarde et restauration

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à `CREATE TABLE`. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à types. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à contraintes. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à clé primaire. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à CREATE TABLE. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Requêtes SELECT

Ce chapitre traite de requêtes select avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Cycle de vie des données



Objectifs pédagogiques

- Comprendre le rôle de requêtes select dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
projection	Comment maîtriser projection dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
filtre	Comment maîtriser filtre dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
tri	Comment maîtriser tri dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
limite	Comment maîtriser limite dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

projection

projection est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. projection aide à rendre ces archives exploitables et sûres.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

filtre

filtre est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. filtre aide à rendre ces archives exploitables et sûres.

tri

tri est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. tri aide à rendre ces archives exploitables et sûres.

Maturité base de données



limite

limite est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. limite aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Requêtes SELECT

```
SELECT c.nom, COUNT(cmd.id) AS total_commandes
FROM clients c
LEFT JOIN commandes cmd ON cmd.client_id = c.id
GROUP BY c.nom
ORDER BY total_commandes DESC;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de requêtes select. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Maturité base de données

SQL



Index



Sécurité



Backup



NoSQL

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser SELECT * dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à projection. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à filtre. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à tri. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à limite. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à projection. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Jointures

Ce chapitre traite de jointures avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

Objectifs pédagogiques

- Comprendre le rôle de jointures dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
INNER JOIN	Comment maîtriser INNER JOIN dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
LEFT JOIN	Comment maîtriser LEFT JOIN dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
relation	Comment maîtriser relation dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
agrégation	Comment maîtriser agrégation dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

INNER JOIN

INNER JOIN est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. INNER JOIN aide à rendre ces archives exploitables et sûres.

Chaîne sauvegarde et restauration**LEFT JOIN**

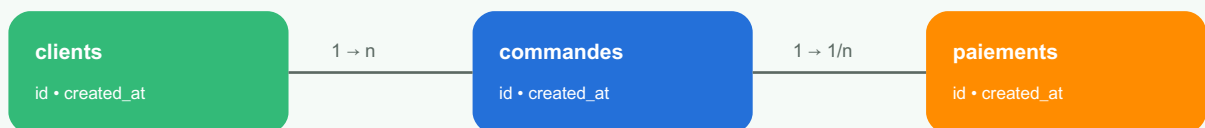
LEFT JOIN est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. LEFT JOIN aide à rendre ces archives exploitables et sûres.

relation

relation est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. relation aide à rendre ces archives exploitables et sûres.

Schéma relationnel simplifié**agrégation**

agrégation est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. agrégation aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

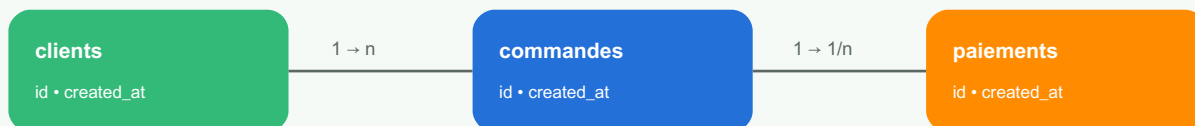
Code — Jointures

```
SELECT c.nom, COUNT(cmd.id) AS total_commandes
FROM clients c
LEFT JOIN commandes cmd ON cmd.client_id = c.id
GROUP BY c.nom
ORDER BY total_commandes DESC;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de jointures. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Schéma relationnel simplifié

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à `INNER JOIN`. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à LEFT JOIN. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à relation. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à agrégation. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à INNER JOIN. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

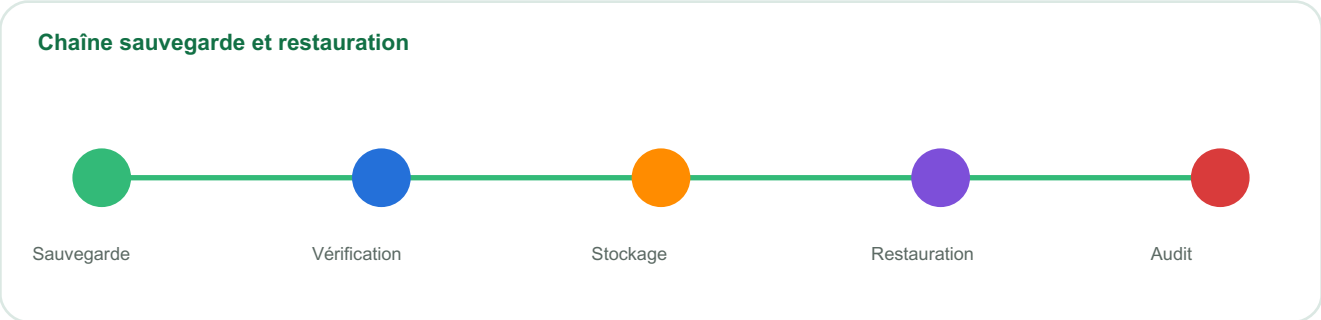
Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Transactions

Ce chapitre traite de transactions avec une approche pratique et professionnelle. L’objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.



Objectifs pédagogiques

- Comprendre le rôle de transactions dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l’audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
BEGIN	Comment maîtriser BEGIN dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
COMMIT	Comment maîtriser COMMIT dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
ROLLBACK	Comment maîtriser ROLLBACK dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
isolation	Comment maîtriser isolation dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

BEGIN

BEGIN est une notion essentielle pour construire une base durable. Une bonne base de données n’est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l’historique d’un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l’organisation perd du temps et prend des risques. BEGIN aide à rendre ces archives exploitables et sûres.

Maturité base de données**COMMIT**

COMMIT est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. COMMIT aide à rendre ces archives exploitables et sûres.

ROLLBACK

ROLLBACK est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. ROLLBACK aide à rendre ces archives exploitables et sûres.

Cycle de vie des données**isolation**

isolation est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. isolation aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Transactions

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de transactions. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Cycle de vie des données

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à `BEGIN`. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à `COMMIT`. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à ROLLBACK. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à isolation. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à BEGIN. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Vues et procédures

Ce chapitre traite de vues et procédures avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Maturité base de données

SQL

Index

Sécurité

Backup

NoSQL

Objectifs pédagogiques

- Comprendre le rôle de vues et procédures dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
vue	Comment maîtriser vue dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
fonction	Comment maîtriser fonction dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
procédure	Comment maîtriser procédure dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
réutilisation	Comment maîtriser réutilisation dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

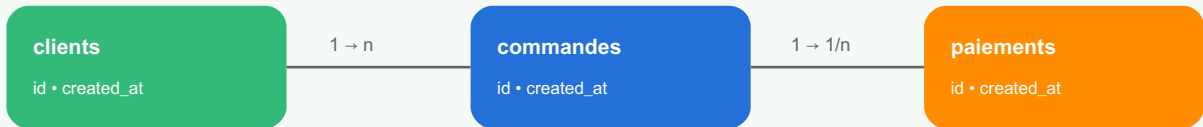
Explication approfondie

vue

vue est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. vue aide à rendre ces archives exploitables et sûres.

Schéma relationnel simplifié



fonction

fonction est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. fonction aide à rendre ces archives exploitables et sûres.

procédure

procédure est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. procédure aide à rendre ces archives exploitables et sûres.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

réutilisation

réutilisation est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. réutilisation aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Vues et procédures

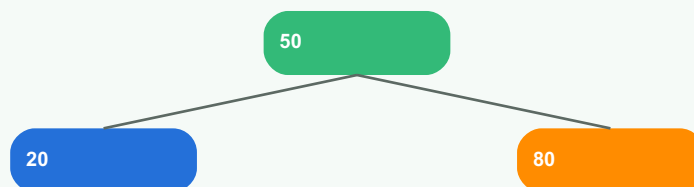
```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de vues et procédures. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à vue. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à fonction. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à procédure. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à réutilisation. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à vue. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

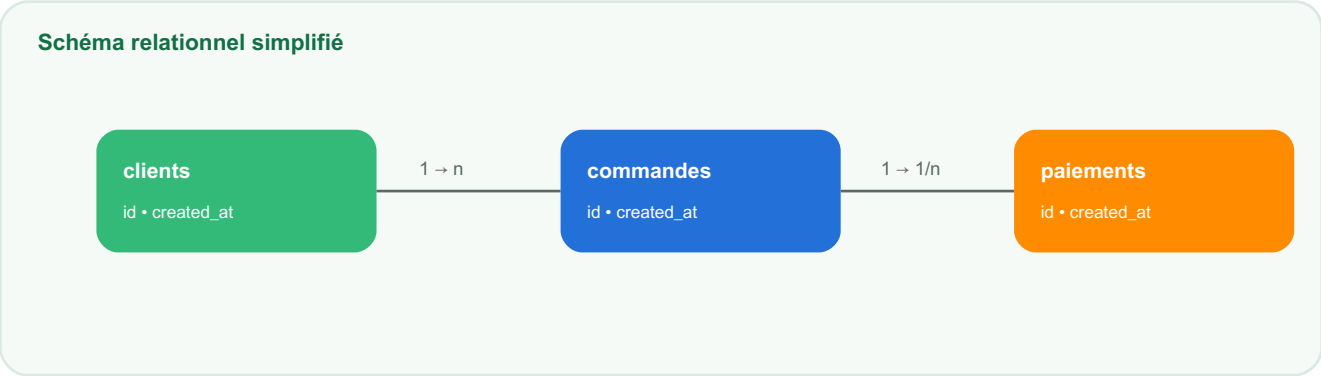
Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Architecture PostgreSQL

Ce chapitre traite de architecture postgresql avec une approche pratique et professionnelle. L’objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.



Objectifs pédagogiques

- Comprendre le rôle de architecture postgresql dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l’audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
schéma	Comment maîtriser schéma dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
rôle	Comment maîtriser rôle dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
extension	Comment maîtriser extension dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
configuration	Comment maîtriser configuration dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

schéma

schéma est une notion essentielle pour construire une base durable. Une bonne base de données n’est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l’historique d’un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l’organisation perd du temps et prend des risques. schéma aide à rendre ces archives exploitables et sûres.

Cycle de vie des données**rôle**

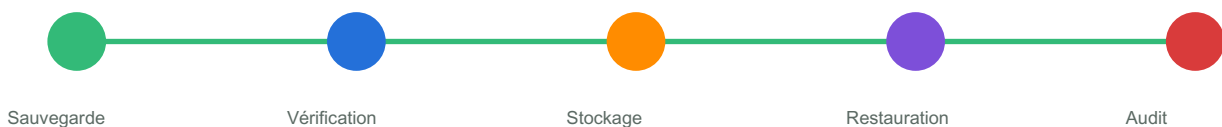
rôle est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. rôle aide à rendre ces archives exploitables et sûres.

extension

extension est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. extension aide à rendre ces archives exploitables et sûres.

Chaîne sauvegarde et restauration**configuration**

configuration est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. configuration aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

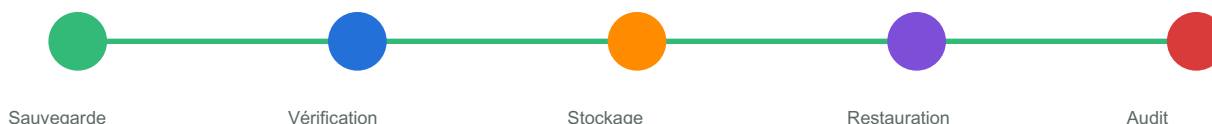
Code — Architecture PostgreSQL

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de architecture postgresql. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Chaîne sauvegarde et restauration

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à schéma. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à rôle. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à extension. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à configuration. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à schéma. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Index PostgreSQL

Ce chapitre traite de index postgresql avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Cycle de vie des données



Objectifs pédagogiques

- Comprendre le rôle de index postgresql dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
B-tree	Comment maîtriser B-tree dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
GIN	Comment maîtriser GIN dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
GiST	Comment maîtriser GiST dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
partial index	Comment maîtriser partial index dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

B-tree

B-tree est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. B-tree aide à rendre ces archives exploitables et sûres.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

GIN

GIN est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. GIN aide à rendre ces archives exploitables et sûres.

GiST

GiST est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. GiST aide à rendre ces archives exploitables et sûres.

Maturité base de données



partial index

partial index est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. partial index aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Index PostgreSQL

```
CREATE INDEX idx_commandes_client_date
ON commandes (client_id, created_at);
```

```
EXPLAIN ANALYZE
SELECT * FROM commandes
WHERE client_id = 10
ORDER BY created_at DESC;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de index postgresql. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Maturité base de données

SQL



Index



Sécurité



Backup



NoSQL

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser SELECT * dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à B-tree. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à GIN. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à GiST. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à partial index. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à B-tree. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

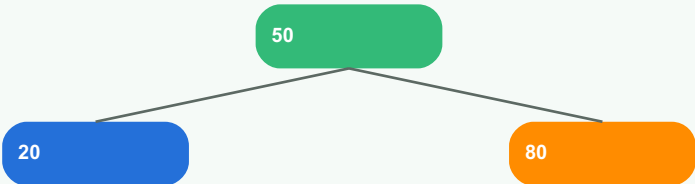
Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

JSONB et données hybrides

Ce chapitre traite de jsonb et données hybrides avec une approche pratique et professionnelle. L’objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Structure logique d’un index



Index B-tree simplifié : réduire le coût de recherche.

Objectifs pédagogiques

- Comprendre le rôle de jsonb et données hybrides dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l’audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
document	Comment maîtriser document dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
requête	Comment maîtriser requête dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
index	Comment maîtriser index dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
flexibilité	Comment maîtriser flexibilité dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

document

document est une notion essentielle pour construire une base durable. Une bonne base de données n’est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l’historique d’un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l’organisation perd du temps et prend des risques. document aide à rendre ces archives exploitables et sûres.

Chaîne sauvegarde et restauration**requête**

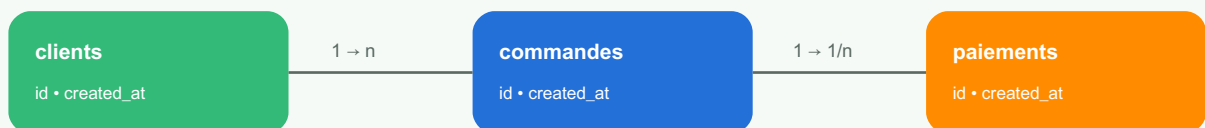
requête est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. requête aide à rendre ces archives exploitables et sûres.

index

index est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. index aide à rendre ces archives exploitables et sûres.

Schéma relationnel simplifié**flexibilité**

flexibilité est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. flexibilité aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

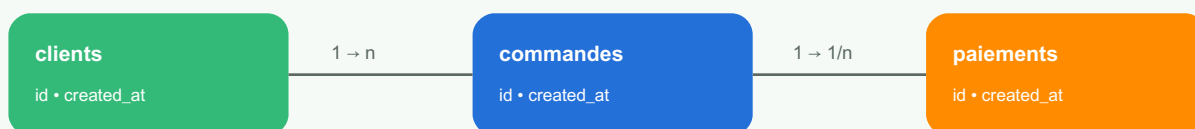
Code — JSONB et données hybrides

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de jsonb et données hybrides. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Schéma relationnel simplifié

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à document. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à requête. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à index. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à flexibilité. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à document. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

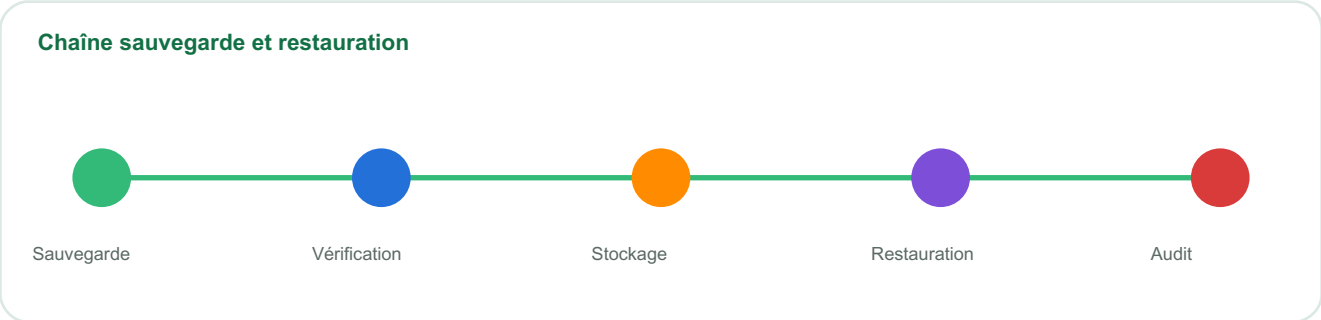
Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Performance et EXPLAIN

Ce chapitre traite de performance et explain avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.



Objectifs pédagogiques

- Comprendre le rôle de performance et explain dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
plan	Comment maîtriser plan dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
coût	Comment maîtriser coût dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
scan	Comment maîtriser scan dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
optimisation	Comment maîtriser optimisation dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

plan

plan est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. plan aide à rendre ces archives exploitables et sûres.

Maturité base de données**coût**

coût est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. coût aide à rendre ces archives exploitables et sûres.

scan

scan est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. scan aide à rendre ces archives exploitables et sûres.

Cycle de vie des données**optimisation**

optimisation est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. optimisation aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Performance et EXPLAIN

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de performance et explain. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Cycle de vie des données

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à plan. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à coût. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à scan. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à optimisation. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à plan. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

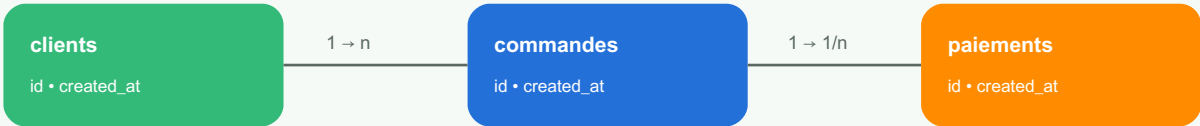
Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Architecture MySQL

Ce chapitre traite de architecture mysql avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Schéma relationnel simplifié



Objectifs pédagogiques

- Comprendre le rôle de architecture mysql dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

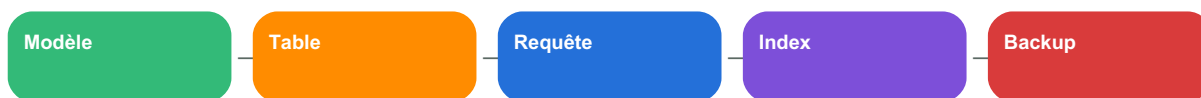
Notion	Question base de données	Livrable attendu
moteur	Comment maîtriser moteur dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
InnoDB	Comment maîtriser InnoDB dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
schéma	Comment maîtriser schéma dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
utilisateur	Comment maîtriser utilisateur dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

moteur

moteur est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. moteur aide à rendre ces archives exploitables et sûres.

Cycle de vie des données**InnoDB**

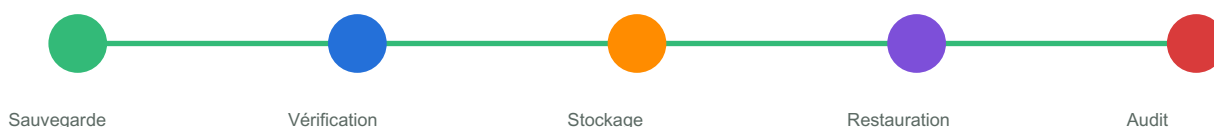
InnoDB est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. InnoDB aide à rendre ces archives exploitables et sûres.

schéma

schéma est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. schéma aide à rendre ces archives exploitables et sûres.

Chaîne sauvegarde et restauration**utilisateur**

utilisateur est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. utilisateur aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

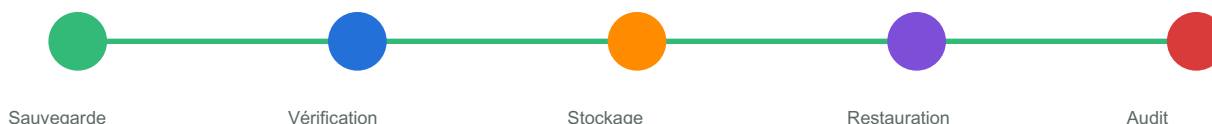
Code — Architecture MySQL

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de architecture mysql. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Chaîne sauvegarde et restauration

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser SELECT * dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à moteur. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à InnoDB. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à schéma. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à utilisateur. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à moteur. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Requêtes et index MySQL

Ce chapitre traite de requêtes et index mysql avec une approche pratique et professionnelle. L’objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Cycle de vie des données



Objectifs pédagogiques

- Comprendre le rôle de requêtes et index mysql dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l’audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
index	Comment maîtriser index dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
jointure	Comment maîtriser jointure dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
EXPLAIN	Comment maîtriser EXPLAIN dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
optimisation	Comment maîtriser optimisation dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

index

index est une notion essentielle pour construire une base durable. Une bonne base de données n’est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l’historique d’un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l’organisation perd du temps et prend des risques. index aide à rendre ces archives exploitables et sûres.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

jointure

jointure est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. jointure aide à rendre ces archives exploitables et sûres.

EXPLAIN

EXPLAIN est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. EXPLAIN aide à rendre ces archives exploitables et sûres.

Maturité base de données



optimisation

optimisation est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. optimisation aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Requêtes et index MySQL

```
CREATE INDEX idx_commandes_client_date
ON commandes (client_id, created_at);
```

```
EXPLAIN ANALYZE
SELECT * FROM commandes
WHERE client_id = 10
ORDER BY created_at DESC;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de requêtes et index mysql. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Maturité base de données

SQL



Index



Sécurité



Backup



NoSQL

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser SELECT * dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à index. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à jointure. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à EXPLAIN. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à optimisation. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à index. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

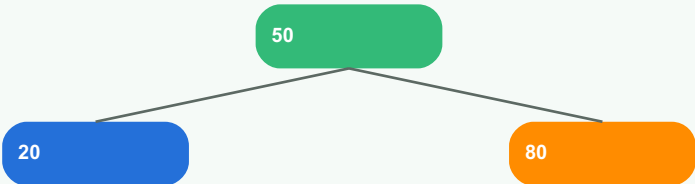
Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Transactions et verrouillage

Ce chapitre traite de transactions et verrouillage avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

Objectifs pédagogiques

- Comprendre le rôle de transactions et verrouillage dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
ACID	Comment maîtriser ACID dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
lock	Comment maîtriser lock dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
deadlock	Comment maîtriser deadlock dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
isolation	Comment maîtriser isolation dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

ACID

ACID est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. ACID aide à rendre ces archives exploitables et sûres.

Chaîne sauvegarde et restauration**lock**

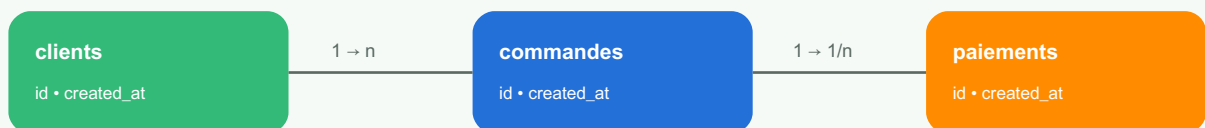
lock est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. lock aide à rendre ces archives exploitables et sûres.

deadlock

deadlock est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. deadlock aide à rendre ces archives exploitables et sûres.

Schéma relationnel simplifié**isolation**

isolation est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. isolation aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

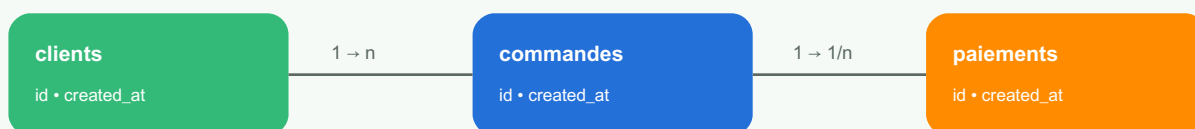
Code — Transactions et verrouillage

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de transactions et verrouillage. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Schéma relationnel simplifié

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à ACID. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à lock. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à deadlock. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à isolation. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à ACID. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

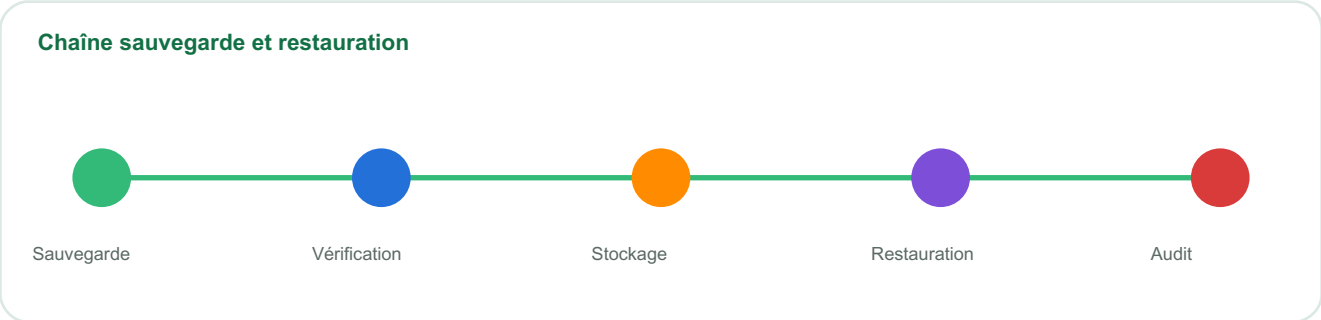
Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Administration MySQL

Ce chapitre traite de administration mysql avec une approche pratique et professionnelle. L’objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.



Objectifs pédagogiques

- Comprendre le rôle de administration mysql dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l’audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
backup	Comment maîtriser backup dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
restore	Comment maîtriser restore dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
réplication	Comment maîtriser réplication dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
monitoring	Comment maîtriser monitoring dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

backup

backup est une notion essentielle pour construire une base durable. Une bonne base de données n’est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l’historique d’un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l’organisation perd du temps et prend des risques. backup aide à rendre ces archives exploitables et sûres.

Maturité base de données**restore**

restore est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. restore aide à rendre ces archives exploitables et sûres.

réplication

réplication est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. réplication aide à rendre ces archives exploitables et sûres.

Cycle de vie des données**monitoring**

monitoring est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. monitoring aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Administration MySQL

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de administration mysql. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Cycle de vie des données

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser SELECT * dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à backup. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à restore. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à réplication. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à monitoring. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à backup. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

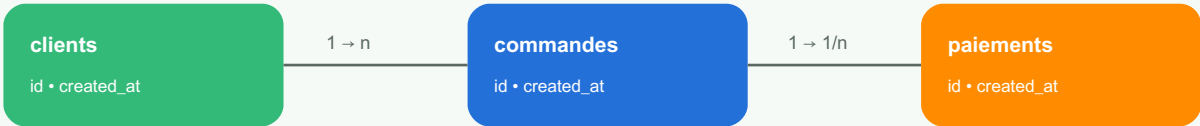
Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Logique documentaire

Ce chapitre traite de logique documentaire avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Schéma relationnel simplifié



Objectifs pédagogiques

- Comprendre le rôle de logique documentaire dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
collection	Comment maîtriser collection dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
document	Comment maîtriser document dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
champ	Comment maîtriser champ dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
embedding	Comment maîtriser embedding dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

collection

collection est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. collection aide à rendre ces archives exploitables et sûres.

Cycle de vie des données**document**

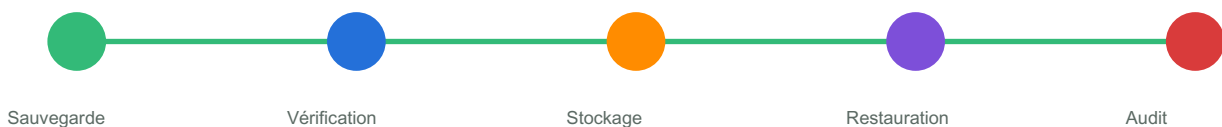
document est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. document aide à rendre ces archives exploitables et sûres.

champ

champ est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. champ aide à rendre ces archives exploitables et sûres.

Chaîne sauvegarde et restauration**embedding**

embedding est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. embedding aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Logique documentaire

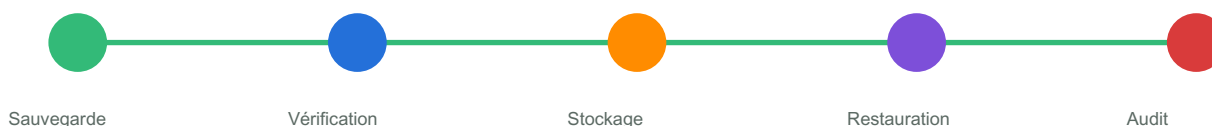
```
db.clients.insertOne({
  nom: "Amina",
  ville: "Kinshasa",
  commandes: [{ reference: "CMD-001", montant: 120 }]
})

db.clients.find({ ville: "Kinshasa" }, { nom: 1, ville: 1 })
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de logique documentaire. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Chaîne sauvegarde et restauration

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à collection. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à document. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à champ. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à embedding. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à collection. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Requêtes MongoDB

Ce chapitre traite de requêtes mongodb avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Cycle de vie des données



Objectifs pédagogiques

- Comprendre le rôle de requêtes mongodb dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
find	Comment maîtriser find dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
filter	Comment maîtriser filter dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
projection	Comment maîtriser projection dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
update	Comment maîtriser update dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

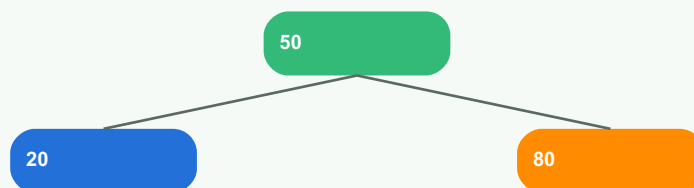
Explication approfondie

find

find est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. find aide à rendre ces archives exploitables et sûres.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

filter

filter est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. filter aide à rendre ces archives exploitables et sûres.

projection

projection est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. projection aide à rendre ces archives exploitables et sûres.

Maturité base de données



update

update est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. update aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Requêtes MongoDB

```
db.clients.insertOne({
  nom: "Amina",
  ville: "Kinshasa",
  commandes: [{ reference: "CMD-001", montant: 120 }]
})

db.clients.find({ ville: "Kinshasa" }, { nom: 1, ville: 1 })
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de requêtes mongodb. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Maturité base de données

SQL



Index



Sécurité



Backup



NoSQL

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser SELECT * dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à find. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à filter. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à projection. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à update. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à find. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Index MongoDB

Ce chapitre traite de index mongodb avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

Objectifs pédagogiques

- Comprendre le rôle de index mongodb dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
single field	Comment maîtriser single field dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
compound	Comment maîtriser compound dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
text	Comment maîtriser text dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
TTL	Comment maîtriser TTL dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

single field

single field est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. single field aide à rendre ces archives exploitables et sûres.

Chaîne sauvegarde et restauration**compound**

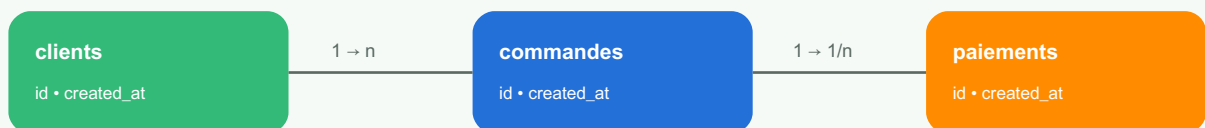
compound est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. compound aide à rendre ces archives exploitables et sûres.

text

text est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. text aide à rendre ces archives exploitables et sûres.

Schéma relationnel simplifié**TTL**

TTL est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. TTL aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Index MongoDB

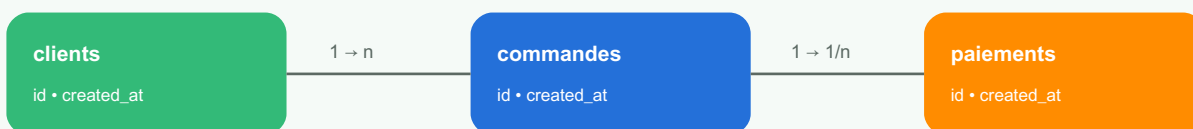
```
CREATE INDEX idx_commandes_client_date
ON commandes (client_id, created_at);

EXPLAIN ANALYZE
SELECT * FROM commandes
WHERE client_id = 10
ORDER BY created_at DESC;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de index mongodb. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Schéma relationnel simplifié

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser SELECT * dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à single field. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à compound. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à text. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à TTL. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à single field. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

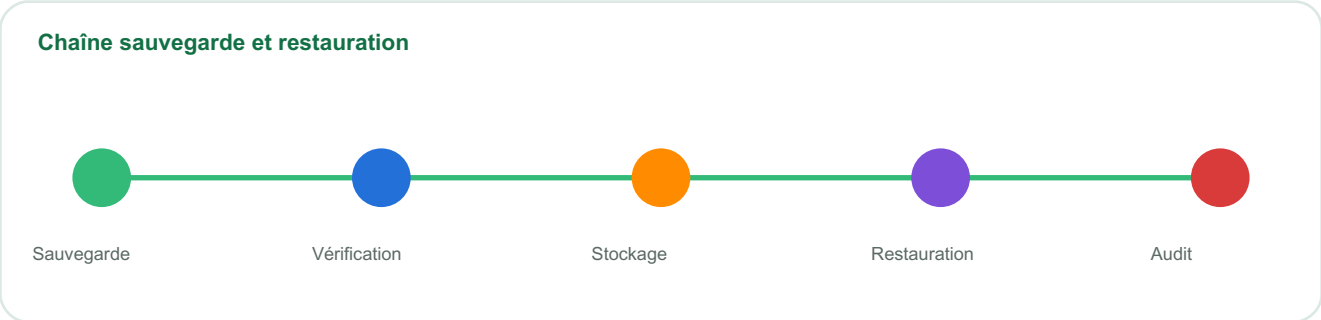
Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Agrégation MongoDB

Ce chapitre traite de agrégation mongodb avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.



Objectifs pédagogiques

- Comprendre le rôle de agrégation mongodb dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
\$match	Comment maîtriser \$match dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
\$group	Comment maîtriser \$group dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
\$project	Comment maîtriser \$project dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
\$lookup	Comment maîtriser \$lookup dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

\$match

\$match est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. \$match aide à rendre ces archives exploitables et sûres.

Maturité base de données**\$group**

\$group est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. \$group aide à rendre ces archives exploitables et sûres.

\$project

\$project est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. \$project aide à rendre ces archives exploitables et sûres.

Cycle de vie des données**\$lookup**

\$lookup est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. \$lookup aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Agrégation MongoDB

```
db.clients.insertOne({
  nom: "Amina",
  ville: "Kinshasa",
  commandes: [{ reference: "CMD-001", montant: 120 }]
})

db.clients.find({ ville: "Kinshasa" }, { nom: 1, ville: 1 })
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de agrégation mongodb. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Cycle de vie des données

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser SELECT * dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à \$match. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à \$group. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à \$project. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à \$lookup. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à \$match. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Choisir SQL ou NoSQL

Ce chapitre traite de choisir sql ou nosql avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Maturité base de données

SQL

Index

Sécurité

Backup

NoSQL

Objectifs pédagogiques

- Comprendre le rôle de choisir sql ou nosql dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
structure	Comment maîtriser structure dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
volume	Comment maîtriser volume dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
flexibilité	Comment maîtriser flexibilité dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
consistance	Comment maîtriser consistance dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

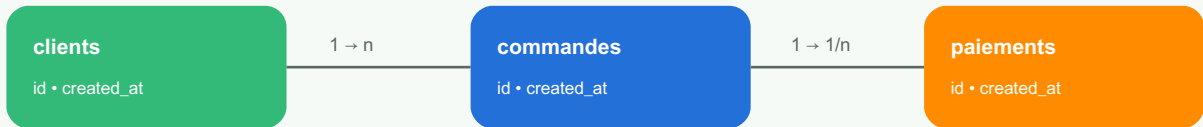
Explication approfondie

structure

structure est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. structure aide à rendre ces archives exploitables et sûres.

Schéma relationnel simplifié

**volume**

volume est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. volume aide à rendre ces archives exploitables et sûres.

flexibilité

flexibilité est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. flexibilité aide à rendre ces archives exploitables et sûres.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

consistance

consistance est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. consistance aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Choisir SQL ou NoSQL

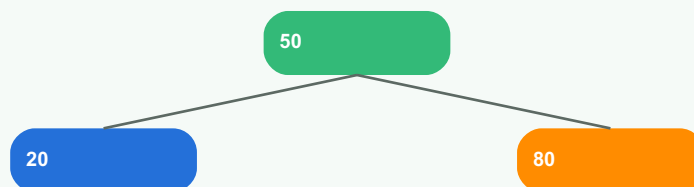
```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de choisir sql ou nosql. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser SELECT * dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à structure. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à volume. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à flexibilité. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à consistance. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à structure. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

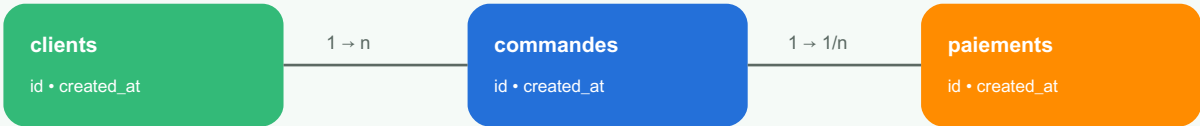
Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Sécurité des accès

Ce chapitre traite de sécurité des accès avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Schéma relationnel simplifié



Objectifs pédagogiques

- Comprendre le rôle de sécurité des accès dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
rôles	Comment maîtriser rôles dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
permissions	Comment maîtriser permissions dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
moindre privilège	Comment maîtriser moindre privilège dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
audit	Comment maîtriser audit dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

rôles

rôles est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. rôles aide à rendre ces archives exploitables et sûres.

Cycle de vie des données**permissions**

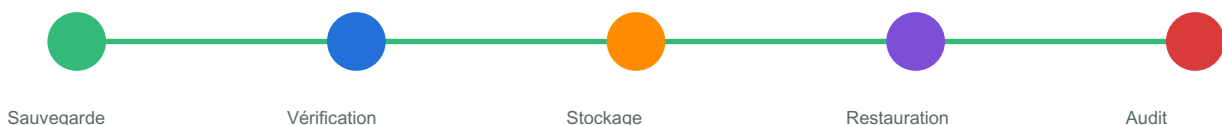
permissions est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. permissions aide à rendre ces archives exploitables et sûres.

moindre privilège

moindre privilège est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. moindre privilège aide à rendre ces archives exploitables et sûres.

Chaîne sauvegarde et restauration**audit**

audit est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. audit aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

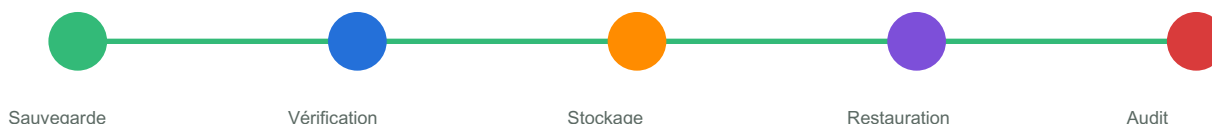
Code — Sécurité des accès

```
CREATE ROLE analyste;
GRANT CONNECT ON DATABASE app_db TO analyste;
GRANT USAGE ON SCHEMA public TO analyste;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO analyste;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de sécurité des accès. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Chaîne sauvegarde et restauration

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à rôles. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à permissions. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à moindre privilège. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à audit. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à rôles. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Protection des données

Ce chapitre traite de protection des données avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Cycle de vie des données



Objectifs pédagogiques

- Comprendre le rôle de protection des données dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
chiffrement	Comment maîtriser chiffrement dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
secrets	Comment maîtriser secrets dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
masquage	Comment maîtriser masquage dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
rétenion	Comment maîtriser rétenion dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

chiffrement

chiffrement est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. chiffrement aide à rendre ces archives exploitables et sûres.

Structure logique d'un index

Index B-tree simplifié : réduire le coût de recherche.

secrets

secrets est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. secrets aide à rendre ces archives exploitables et sûres.

masquage

masquage est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. masquage aide à rendre ces archives exploitables et sûres.

Maturité base de données**rétenion**

rétenion est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. rétenion aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Protection des données

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de protection des données. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Maturité base de données

SQL



Index



Sécurité



Backup



NoSQL

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser SELECT * dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à chiffrement. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à secrets. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à masquage. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à rétention. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à chiffrement. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

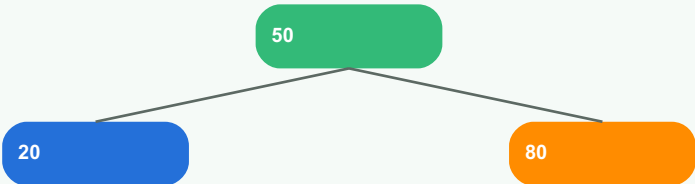
Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Sauvegarde et restauration

Ce chapitre traite de sauvegarde et restauration avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

Objectifs pédagogiques

- Comprendre le rôle de sauvegarde et restauration dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
RPO	Comment maîtriser RPO dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
RTO	Comment maîtriser RTO dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
dump	Comment maîtriser dump dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
test restore	Comment maîtriser test restore dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

RPO

RPO est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. RPO aide à rendre ces archives exploitables et sûres.

Chaîne sauvegarde et restauration**RTO**

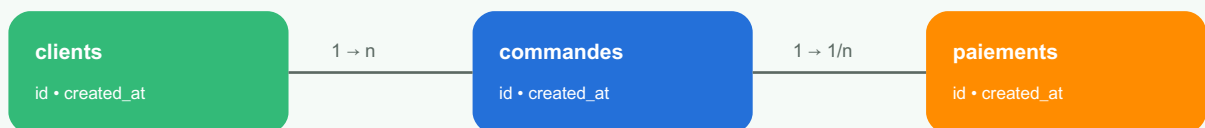
RTO est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. RTO aide à rendre ces archives exploitables et sûres.

dump

dump est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. dump aide à rendre ces archives exploitables et sûres.

Schéma relationnel simplifié**test restore**

test restore est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. test restore aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Sauvegarde et restauration

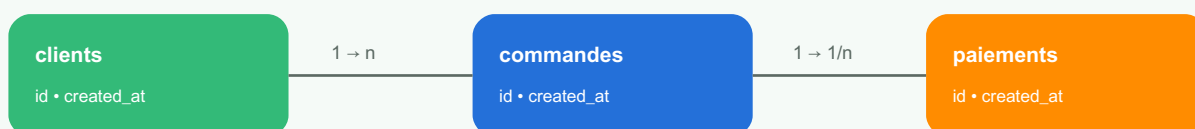
```
# PostgreSQL
pg_dump -Fc app_db > app_db.backup
pg_restore -d app_db_restore app_db.backup

# MySQL
mysqldump -u user -p app_db > app_db.sql
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de sauvegarde et restauration. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Schéma relationnel simplifié

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à RPO. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à RTO. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à dump. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à test restore. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à RPO. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

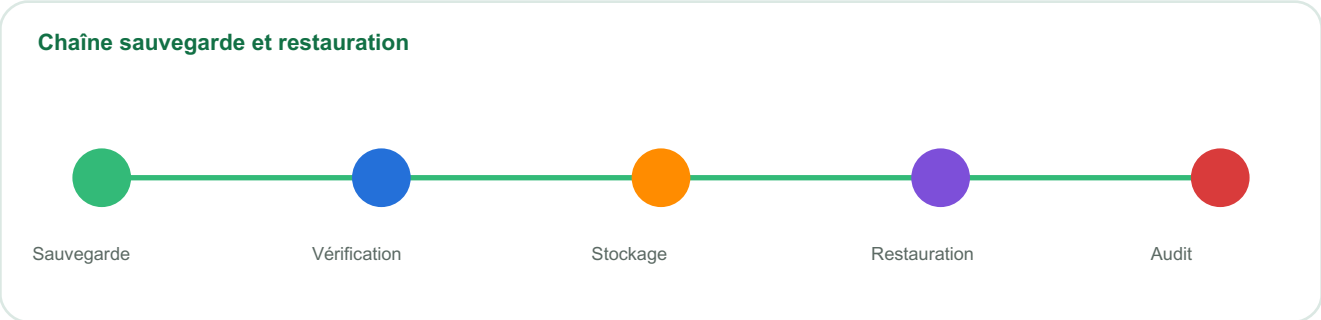
Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Monitoring et maintenance

Ce chapitre traite de monitoring et maintenance avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.



Objectifs pédagogiques

- Comprendre le rôle de monitoring et maintenance dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
logs	Comment maîtriser logs dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
métriques	Comment maîtriser métriques dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
alertes	Comment maîtriser alertes dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
vacuum	Comment maîtriser vacuum dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

logs

logs est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. logs aide à rendre ces archives exploitables et sûres.

Maturité base de données**métriques**

métriques est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. métriques aide à rendre ces archives exploitables et sûres.

alertes

alertes est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. alertes aide à rendre ces archives exploitables et sûres.

Cycle de vie des données**vacuum**

vacuum est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. vacuum aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Monitoring et maintenance

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de monitoring et maintenance. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Cycle de vie des données

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à logs. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à métriques. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à alertes. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à vacuum. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à logs. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Migration et versioning

Ce chapitre traite de migration et versioning avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Maturité base de données

SQL

Index

Sécurité

Backup

NoSQL

Objectifs pédagogiques

- Comprendre le rôle de migration et versioning dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

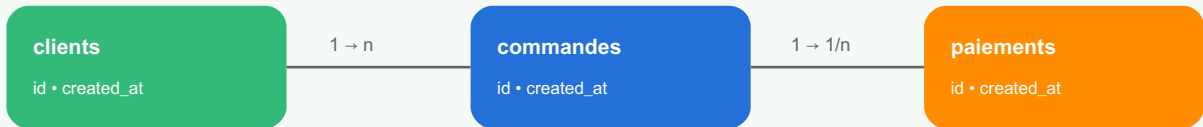
Notion	Question base de données	Livrable attendu
schema migration	Comment maîtriser schema migration dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
rollback	Comment maîtriser rollback dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
compatibilité	Comment maîtriser compatibilité dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
CI/CD	Comment maîtriser CI/CD dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

schema migration

schema migration est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. schema migration aide à rendre ces archives exploitables et sûres.

Schéma relationnel simplifié**rollback**

rollback est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. rollback aide à rendre ces archives exploitables et sûres.

compatibilité

compatibilité est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. compatibilité aide à rendre ces archives exploitables et sûres.

Structure logique d'un index

Index B-tree simplifié : réduire le coût de recherche.

CI/CD

CI/CD est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. CI/CD aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Migration et versioning

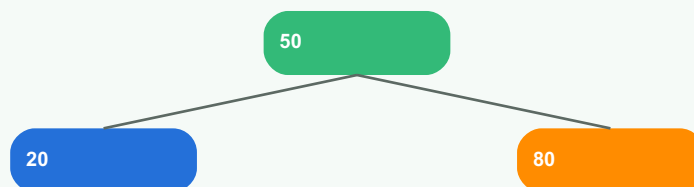
```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de migration et versioning. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à schéma migration. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à rollback. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à compatibilité. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à CI/CD. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à schema migration. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

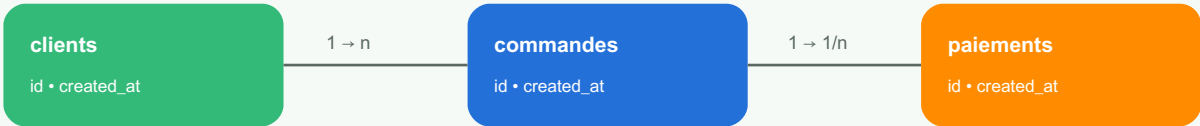
Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Base e-commerce

Ce chapitre traite de base e-commerce avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Schéma relationnel simplifié



Objectifs pédagogiques

- Comprendre le rôle de base e-commerce dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
clients	Comment maîtriser clients dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
produits	Comment maîtriser produits dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
commandes	Comment maîtriser commandes dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
paiements	Comment maîtriser paiements dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

clients

clients est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. clients aide à rendre ces archives exploitables et sûres.

Cycle de vie des données**produits**

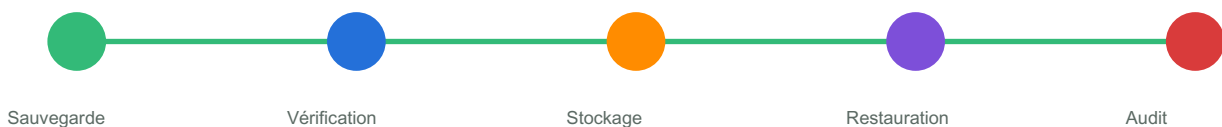
produits est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. produits aide à rendre ces archives exploitables et sûres.

commandes

commandes est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. commandes aide à rendre ces archives exploitables et sûres.

Chaîne sauvegarde et restauration**paiements**

paiements est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. paiements aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

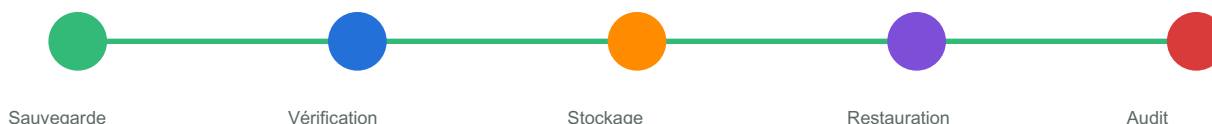
Code — Base e-commerce

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de base e-commerce. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Chaîne sauvegarde et restauration

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à clients. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à produits. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à commandes. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à paiements. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à clients. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Base hospitalière

Ce chapitre traite de base hospitalière avec une approche pratique et professionnelle. L’objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Cycle de vie des données



Objectifs pédagogiques

- Comprendre le rôle de base hospitalière dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l’audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
patients	Comment maîtriser patients dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
consultations	Comment maîtriser consultations dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
prescriptions	Comment maîtriser prescriptions dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
confidentialité	Comment maîtriser confidentialité dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

patients

patients est une notion essentielle pour construire une base durable. Une bonne base de données n’est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l’historique d’un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l’organisation perd du temps et prend des risques. patients aide à rendre ces archives exploitables et sûres.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

consultations

consultations est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. consultations aide à rendre ces archives exploitables et sûres.

prescriptions

prescriptions est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. prescriptions aide à rendre ces archives exploitables et sûres.

Maturité base de données



confidentialité

confidentialité est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. confidentialité aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Base hospitalière

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de base hospitalière. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Maturité base de données

SQL



Index



Sécurité



Backup



NoSQL

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser SELECT * dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à patients. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à consultations. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à prescriptions. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à confidentialité. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à patients. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

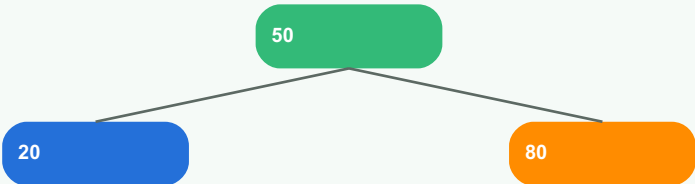
Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Base financière

Ce chapitre traite de base financière avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

Objectifs pédagogiques

- Comprendre le rôle de base financière dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
comptes	Comment maîtriser comptes dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
transactions	Comment maîtriser transactions dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
audit	Comment maîtriser audit dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
fraude	Comment maîtriser fraude dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

comptes

comptes est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. comptes aide à rendre ces archives exploitables et sûres.

Chaîne sauvegarde et restauration**transactions**

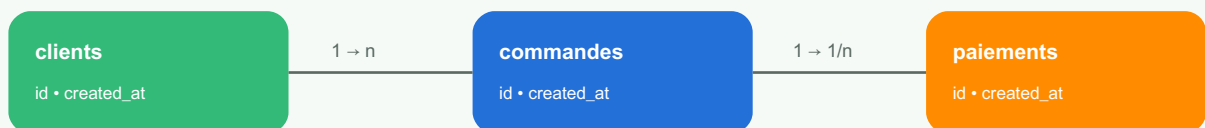
transactions est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. transactions aide à rendre ces archives exploitables et sûres.

audit

audit est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. audit aide à rendre ces archives exploitables et sûres.

Schéma relationnel simplifié**fraude**

fraude est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. fraude aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

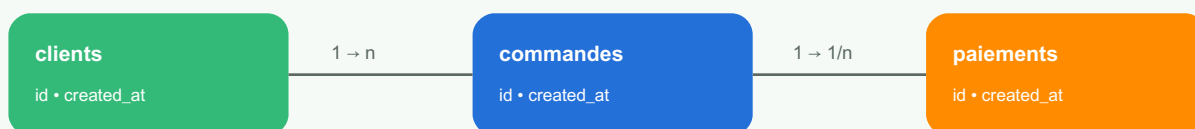
Code — Base financière

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de base financière. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Schéma relationnel simplifié

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à comptes. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à transactions. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à audit. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à fraude. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à comptes. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

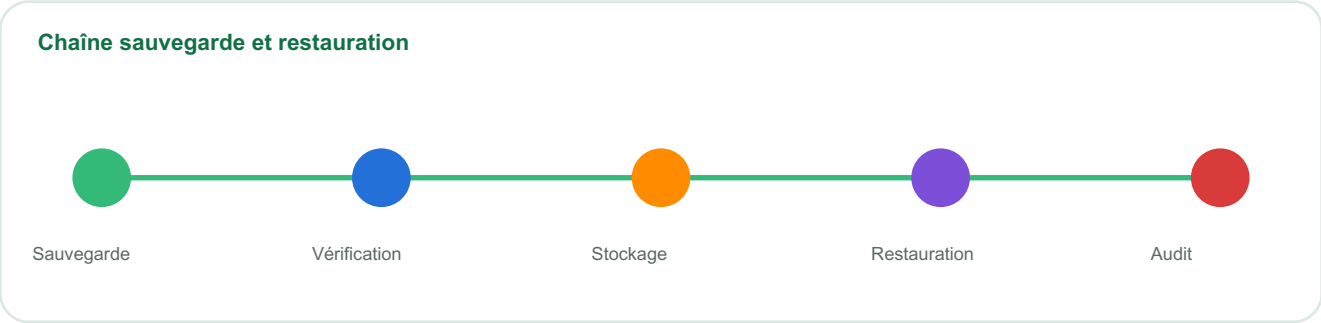
Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Base logistique

Ce chapitre traite de base logistique avec une approche pratique et professionnelle. L’objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.



Objectifs pédagogiques

- Comprendre le rôle de base logistique dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l’audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
colis	Comment maîtriser colis dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
agences	Comment maîtriser agences dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
tracking	Comment maîtriser tracking dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
preuves	Comment maîtriser preuves dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

colis

colis est une notion essentielle pour construire une base durable. Une bonne base de données n’est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l’historique d’un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l’organisation perd du temps et prend des risques. colis aide à rendre ces archives exploitables et sûres.

Maturité base de données



agences

agences est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. agences aide à rendre ces archives exploitables et sûres.

tracking

tracking est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. tracking aide à rendre ces archives exploitables et sûres.

Cycle de vie des données



preuves

preuves est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. preuves aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Base logistique

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de base logistique. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Cycle de vie des données

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à colis. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à agences. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à tracking. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à preuves. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à colis. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Base analytique

Ce chapitre traite de base analytique avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Maturité base de données

SQL

Index

Sécurité

Backup

NoSQL

Objectifs pédagogiques

- Comprendre le rôle de base analytique dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

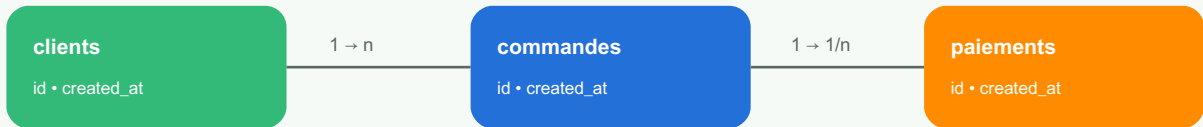
Notion	Question base de données	Livrable attendu
faits	Comment maîtriser faits dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
dimensions	Comment maîtriser dimensions dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
KPI	Comment maîtriser KPI dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
dashboard	Comment maîtriser dashboard dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

faits

faits est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. faits aide à rendre ces archives exploitables et sûres.

Schéma relationnel simplifié**dimensions**

dimensions est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. dimensions aide à rendre ces archives exploitables et sûres.

KPI

KPI est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. KPI aide à rendre ces archives exploitables et sûres.

Structure logique d'un index

Index B-tree simplifié : réduire le coût de recherche.

dashboard

dashboard est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. dashboard aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Base analytique

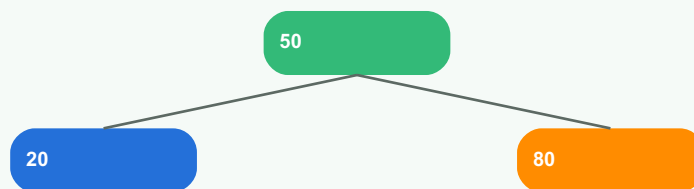
```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de base analytique. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à faits. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à dimensions. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à KPI. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à dashboard. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à faits. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

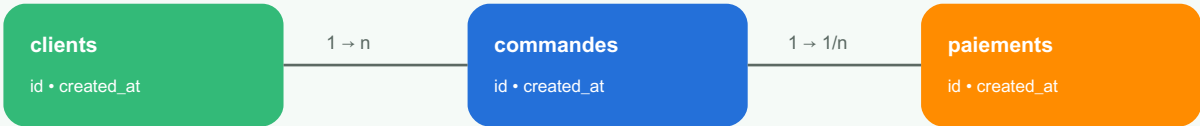
Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Banque d'exercices

Ce chapitre traite de banque d'exercices avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Schéma relationnel simplifié



Objectifs pédagogiques

- Comprendre le rôle de banque d'exercices dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
SQL	Comment maîtriser SQL dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
relations	Comment maîtriser relations dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
index	Comment maîtriser index dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
sécurité	Comment maîtriser sécurité dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

SQL

SQL est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. SQL aide à rendre ces archives exploitables et sûres.

Cycle de vie des données**relations**

relations est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. relations aide à rendre ces archives exploitables et sûres.

index

index est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. index aide à rendre ces archives exploitables et sûres.

Chaîne sauvegarde et restauration**sécurité**

sécurité est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. sécurité aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

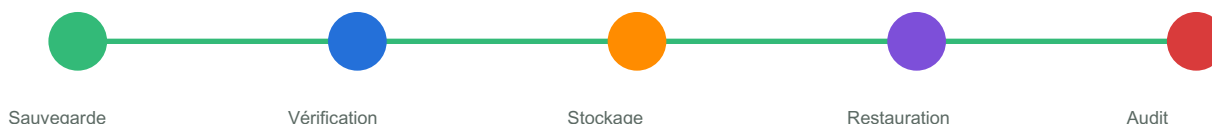
Code — Banque d'exercices

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de banque d'exercices. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Chaîne sauvegarde et restauration

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à SQL. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à relations. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à index. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à sécurité. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à SQL. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Examens blancs

Ce chapitre traite de examens blancs avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Cycle de vie des données



Objectifs pédagogiques

- Comprendre le rôle de examens blancs dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
requêtes	Comment maîtriser requêtes dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
modélisation	Comment maîtriser modélisation dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
administration	Comment maîtriser administration dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
projet	Comment maîtriser projet dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

requêtes

requêtes est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. requêtes aide à rendre ces archives exploitables et sûres.

Structure logique d'un index

Index B-tree simplifié : réduire le coût de recherche.

modélisation

modélisation est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. modélisation aide à rendre ces archives exploitables et sûres.

administration

administration est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. administration aide à rendre ces archives exploitables et sûres.

Maturité base de données**projet**

projet est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. projet aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

Code — Examens blancs

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de examens blancs. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Maturité base de données

SQL



Index



Sécurité



Backup



NoSQL

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser SELECT * dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à requêtes. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à modélisation. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à administration. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à projet. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à requêtes. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Glossaire et feuilles de route

Ce chapitre traite de glossaire et feuilles de route avec une approche pratique et professionnelle. L'objectif est de comprendre la logique, écrire des requêtes fiables, anticiper les erreurs, documenter les choix et protéger les données dans un vrai système.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

Objectifs pédagogiques

- Comprendre le rôle de glossaire et feuilles de route dans une application réelle.
- Concevoir ou interroger une base avec rigueur.
- Identifier les relations, contraintes et impacts de performance.
- Prévoir la sécurité, l'audit et la sauvegarde.
- Produire un livrable clair : schéma, requête, documentation ou procédure.

Notion	Question base de données	Livrable attendu
termes	Comment maîtriser termes dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
références	Comment maîtriser références dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
progression	Comment maîtriser progression dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.
DBA	Comment maîtriser DBA dans un système fiable ?	Règle claire, requête testée, contrainte ou procédure documentée.

Explication approfondie

termes

termes est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. termes aide à rendre ces archives exploitables et sûres.

Chaîne sauvegarde et restauration**références**

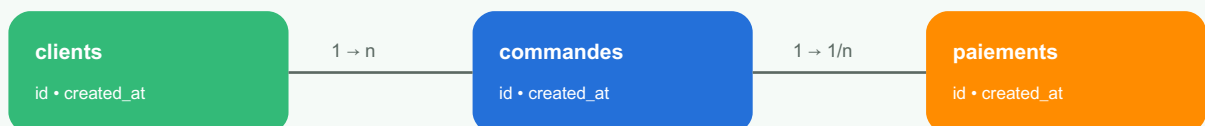
références est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. références aide à rendre ces archives exploitables et sûres.

progression

progression est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. progression aide à rendre ces archives exploitables et sûres.

Schéma relationnel simplifié**DBA**

DBA est une notion essentielle pour construire une base durable. Une bonne base de données n'est pas seulement un lieu de stockage : elle exprime les règles métier, protège les informations, accélère les recherches et permet aux équipes de comprendre l'historique d'un système.

Analogie : une base de données ressemble à des archives professionnelles. Si les dossiers sont mal classés, sans index, sans autorisation et sans sauvegarde, l'organisation perd du temps et prend des risques. DBA aide à rendre ces archives exploitables et sûres.

Exemple de code ou requête

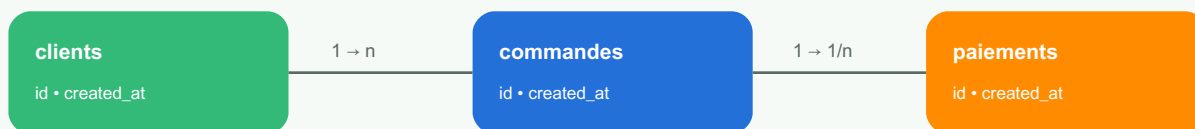
Code — Glossaire et feuilles de route

```
BEGIN;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

L'exemple montre une pratique directement exploitable. Dans un projet réel, chaque requête doit être testée, documentée et reliée à un besoin métier précis. Les changements de schéma doivent être versionnés.

Cas pratique professionnel

Une application de paiement, de santé, d'e-commerce ou de logistique dépend fortement de glossaire et feuilles de route. Une mauvaise conception peut créer des incohérences, des lenteurs, des pertes de données ou des failles d'accès. Une bonne conception rend le système robuste et auditable.

Schéma relationnel simplifié

Bonnes pratiques

- Nommer les tables et colonnes avec un vocabulaire métier stable.
- Utiliser des clés primaires et étrangères quand la relation est réelle.
- Créer des index pour les requêtes fréquentes, pas au hasard.
- Tester les sauvegardes par des restaurations régulières.
- Appliquer le moindre privilège pour les accès.
- Documenter les migrations et les règles de conservation des données.

Pièges à éviter

- Créer une table sans comprendre le besoin métier.
- Utiliser `SELECT *` dans des traitements critiques.
- Multiplier les index sans mesurer leur impact.
- Oublier les contraintes d'unicité et d'intégrité.
- Faire des sauvegardes sans jamais tester la restauration.

Exercices corrigés

Exercice 1 : proposez une solution liée à termes. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 1 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 2 : proposez une solution liée à références. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 2 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 3 : proposez une solution liée à progression. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 3 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 4 : proposez une solution liée à DBA. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 4 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Exercice 5 : proposez une solution liée à termes. Écrivez le schéma ou la requête, expliquez le choix et indiquez un risque.

Corrigé 5 : une réponse solide contient une contrainte claire, une requête testable, une justification d'index ou de relation, et une note sur la sécurité ou la sauvegarde.

Quiz de validation

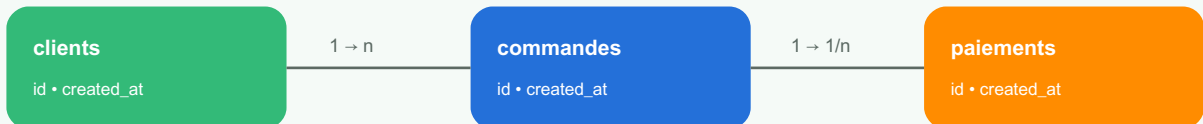
- Quelle règle métier est représentée dans le schéma ?
- Quelle requête doit être optimisée en priorité ?
- Quel index est utile et pourquoi ?
- Quel accès faut-il limiter ?
- Comment restaurer les données en cas d'incident ?

Banque de 420 exercices bases de données

Cette banque entraîne les apprenants à modéliser, écrire du SQL, créer des index, sécuriser les accès, sauvegarder, restaurer et choisir entre SQL et NoSQL.

Modélisation

Schéma relationnel simplifié



Exercice 1 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 2 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 3 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 4 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 5 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 6 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 7 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 8 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 9 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 10 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 11 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 12 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 13 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 14 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 15 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 17 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 18 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 19 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 20 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 21 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 22 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 23 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 24 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 25 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 26 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 27 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 28 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 29 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 30 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 31 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 32 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 33 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 34 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 35 : réalisez une tâche liée à Modélisation. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Cycle de vie des données

```
graph LR; A[Modèle] --> B[Table]; B --> C[Requête]; C --> D[Index]; D --> E[Backup];
```

The diagram illustrates the data lifecycle as a sequence of five steps, each represented by a colored rounded rectangle connected by arrows:

- Modèle** (Green)
- Table** (Orange)
- Requête** (Blue)
- Index** (Purple)
- Backup** (Red)

Exercice 59 : réalisez une tâche liée à SQL SELECT. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 60 : réalisez une tâche liée à SQL SELECT. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 61 : réalisez une tâche liée à SQL SELECT. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 62 : réalisez une tâche liée à SQL SELECT. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 63 : réalisez une tâche liée à SQL SELECT. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 64 : réalisez une tâche liée à SQL SELECT. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 65 : réalisez une tâche liée à SQL SELECT. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 66 : réalisez une tâche liée à SQL SELECT. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 67 : réalisez une tâche liée à SQL SELECT. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 68 : réalisez une tâche liée à SQL SELECT. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 69 : réalisez une tâche liée à SQL SELECT. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 70 : réalisez une tâche liée à SQL SELECT. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Jointures

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

Exercice 71 : réalisez une tâche liée à Jointures. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 72 : réalisez une tâche liée à Jointures. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 73 : réalisez une tâche liée à Jointures. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 74 : réalisez une tâche liée à Jointures. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 75 : réalisez une tâche liée à Jointures. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 76 : réalisez une tâche liée à Jointures. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 77 : réalisez une tâche liée à Jointures. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 101 : réalisez une tâche liée à Jointures. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 102 : réalisez une tâche liée à Jointures. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

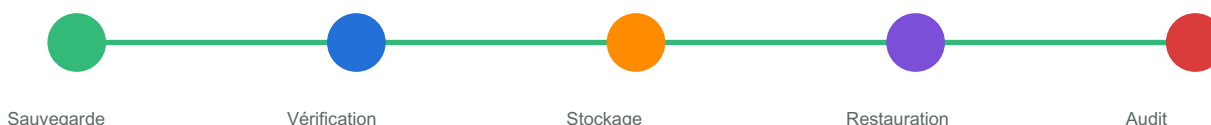
Exercice 103 : réalisez une tâche liée à Jointures. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 104 : réalisez une tâche liée à Jointures. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 105 : réalisez une tâche liée à Jointures. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Transactions

Chaîne sauvegarde et restauration



Exercice 106 : réalisez une tâche liée à Transactions. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 107 : réalisez une tâche liée à Transactions. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 108 : réalisez une tâche liée à Transactions. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 109 : réalisez une tâche liée à Transactions. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 110 : réalisez une tâche liée à Transactions. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 111 : réalisez une tâche liée à Transactions. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 112 : réalisez une tâche liée à Transactions. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 113 : réalisez une tâche liée à Transactions. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 114 : réalisez une tâche liée à Transactions. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 115 : réalisez une tâche liée à Transactions. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 116 : réalisez une tâche liée à Transactions. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 117 : réalisez une tâche liée à Transactions. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 118 : réalisez une tâche liée à Transactions. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 119 : réalisez une tâche liée à Transactions. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 120 : réalisez une tâche liée à Transactions. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 140 : réalisez une tâche liée à Transactions. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

SQL



Index



Sécurité



Backup



NoSQL

Exercice 159 : réalisez une tâche liée à PostgreSQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 160 : réalisez une tâche liée à PostgreSQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 161 : réalisez une tâche liée à PostgreSQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 162 : réalisez une tâche liée à PostgreSQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 163 : réalisez une tâche liée à PostgreSQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 164 : réalisez une tâche liée à PostgreSQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 165 : réalisez une tâche liée à PostgreSQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 166 : réalisez une tâche liée à PostgreSQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 167 : réalisez une tâche liée à PostgreSQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 168 : réalisez une tâche liée à PostgreSQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 169 : réalisez une tâche liée à PostgreSQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 170 : réalisez une tâche liée à PostgreSQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 171 : réalisez une tâche liée à PostgreSQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 172 : réalisez une tâche liée à PostgreSQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

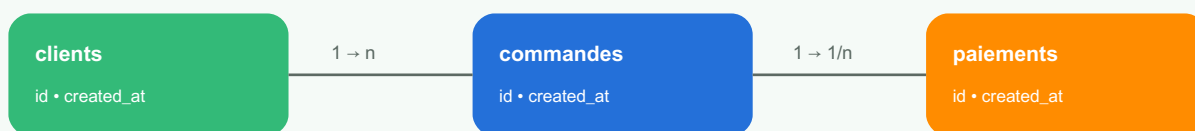
Exercice 173 : réalisez une tâche liée à PostgreSQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 174 : réalisez une tâche liée à PostgreSQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 175 : réalisez une tâche liée à PostgreSQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

MySQL

Schéma relationnel simplifié



Exercice 176 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 177 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 179 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 181 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 183 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 185 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 187 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 189 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 191 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 193 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 195 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 197 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 199 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 201 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 202 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 203 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 204 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 205 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 206 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 207 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 208 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 209 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 210 : réalisez une tâche liée à MySQL. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

MongoDB

Cycle de vie des données



Exercice 211 : réalisez une tâche liée à MongoDB. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 212 : réalisez une tâche liée à MongoDB. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 213 : réalisez une tâche liée à MongoDB. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 214 : réalisez une tâche liée à MongoDB. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 215 : réalisez une tâche liée à MongoDB. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 216 : réalisez une tâche liée à MongoDB. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 217 : réalisez une tâche liée à MongoDB. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 218 : réalisez une tâche liée à MongoDB. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 219 : réalisez une tâche liée à MongoDB. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

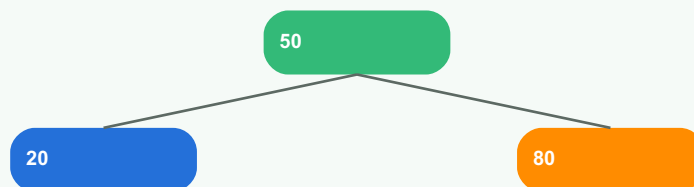
Exercice 220 : réalisez une tâche liée à MongoDB. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 221 : réalisez une tâche liée à MongoDB. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 245 : réalisez une tâche liée à MongoDB. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Index

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

Exercice 246 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 247 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 248 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 249 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 250 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 251 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 252 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 253 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 254 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 255 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 256 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 257 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 258 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 259 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 260 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 261 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 262 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 263 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 264 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 265 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 266 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 267 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 268 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 269 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 270 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 271 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 272 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 273 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 274 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 275 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 276 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 277 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

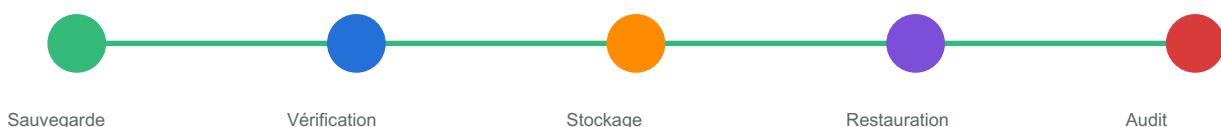
Exercice 278 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 279 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 280 : réalisez une tâche liée à Index. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Sécurité

Chaîne sauvegarde et restauration



Exercice 281 : réalisez une tâche liée à Sécurité. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 282 : réalisez une tâche liée à Sécurité. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 307 : réalisez une tâche liée à Sécurité. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 308 : réalisez une tâche liée à Sécurité. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 309 : réalisez une tâche liée à Sécurité. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 310 : réalisez une tâche liée à Sécurité. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 311 : réalisez une tâche liée à Sécurité. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 312 : réalisez une tâche liée à Sécurité. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 313 : réalisez une tâche liée à Sécurité. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 314 : réalisez une tâche liée à Sécurité. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 315 : réalisez une tâche liée à Sécurité. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Sauvegarde

Maturité base de données



Exercice 316 : réalisez une tâche liée à Sauvegarde. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 317 : réalisez une tâche liée à Sauvegarde. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 318 : réalisez une tâche liée à Sauvegarde. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 319 : réalisez une tâche liée à Sauvegarde. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 320 : réalisez une tâche liée à Sauvegarde. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 321 : réalisez une tâche liée à Sauvegarde. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 322 : réalisez une tâche liée à Sauvegarde. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 323 : réalisez une tâche liée à Sauvegarde. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 324 : réalisez une tâche liée à Sauvegarde. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

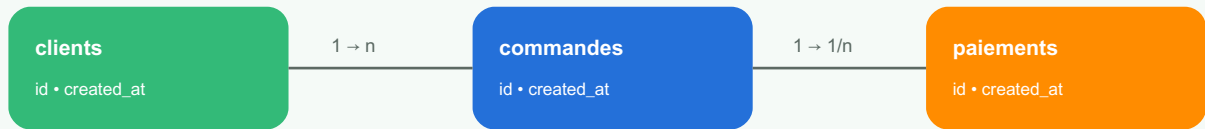
Exercice 325 : réalisez une tâche liée à Sauvegarde. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 349 : réalisez une tâche liée à Sauvegarde. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 350 : réalisez une tâche liée à Sauvegarde. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Migration

Schéma relationnel simplifié



Exercice 351 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 352 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 353 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 354 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 355 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 356 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 357 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 358 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 359 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 360 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 361 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 362 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 363 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 364 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 365 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 366 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 367 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 385 : réalisez une tâche liée à Migration. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 389 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 391 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 393 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 395 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 397 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 399 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 401 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 403 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 405 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, svntaxe correcte, justification et risque identifié.

Exercice 407 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 409 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 411 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure. Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 412 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 413 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 414 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 415 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 416 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 417 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 418 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 419 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Exercice 420 : réalisez une tâche liée à Projets. Produisez un schéma, une requête, une contrainte, un index ou une procédure.
Corrigé attendu : logique métier, syntaxe correcte, justification et risque identifié.

Examens blancs

Examen blanc 1

- Partie A : modélisation relationnelle.
- Partie B : requêtes SQL et jointures.
- Partie C : index et performance.
- Partie D : sécurité et sauvegarde.
- Partie E : projet complet avec PostgreSQL, MySQL ou MongoDB.

Examen blanc 2

- Partie A : modélisation relationnelle.
- Partie B : requêtes SQL et jointures.
- Partie C : index et performance.
- Partie D : sécurité et sauvegarde.
- Partie E : projet complet avec PostgreSQL, MySQL ou MongoDB.

Examen blanc 3

- Partie A : modélisation relationnelle.
- Partie B : requêtes SQL et jointures.
- Partie C : index et performance.
- Partie D : sécurité et sauvegarde.
- Partie E : projet complet avec PostgreSQL, MySQL ou MongoDB.

Examen blanc 4

- Partie A : modélisation relationnelle.
- Partie B : requêtes SQL et jointures.

- Partie C : index et performance.
- Partie D : sécurité et sauvegarde.
- Partie E : projet complet avec PostgreSQL, MySQL ou MongoDB.

Examen blanc 5

- Partie A : modélisation relationnelle.
- Partie B : requêtes SQL et jointures.
- Partie C : index et performance.
- Partie D : sécurité et sauvegarde.
- Partie E : projet complet avec PostgreSQL, MySQL ou MongoDB.

Examen blanc 6

- Partie A : modélisation relationnelle.
- Partie B : requêtes SQL et jointures.
- Partie C : index et performance.
- Partie D : sécurité et sauvegarde.
- Partie E : projet complet avec PostgreSQL, MySQL ou MongoDB.

Glossaire bases de données

Terme	Définition
Table	Structure relationnelle composée de colonnes et de lignes.
Clé primaire	Identifiant unique d'une ligne.
Clé étrangère	Lien entre deux tables.
Index	Structure accélérant certaines recherches.
Transaction	Ensemble d'opérations validées ou annulées ensemble.
Normalisation	Méthode de réduction des redondances et incohérences.
Collection	Regroupement de documents dans MongoDB.
RPO/RTO	Objectifs de perte maximale et délai de restauration.

Feuille de route DBA et développeur backend

- Comprendre la modélisation relationnelle.
- Maîtriser SQL, jointures, agrégations et transactions.
- Apprendre PostgreSQL, MySQL et MongoDB.

- Optimiser avec index et plans d'exécution.
- Sécuriser les accès et protéger les données.
- Automatiser sauvegardes, restaurations et migrations.
- Construire des projets complets avec audit et monitoring.

Contact professionnel

Charmant Nyungu — Consultant en innovation technologique, transformation numérique, cybersécurité, intelligence artificielle, développement logiciel et stratégie digitale.

- www.charmantnyungu.com
- consultant@charmantnyungu.com
- +243 835 137 837

Fiche pratique base de données 167

Cette fiche sert de révision guidée. L'apprenant doit choisir un besoin métier, proposer un modèle, écrire une requête, prévoir un index, appliquer une règle de sécurité et définir une stratégie de sauvegarde.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

Étape	Résultat attendu
Modèle	Tables, collections, relations ou documents.
Requête	Résultat exact, filtré et performant.
Index	Justification selon la requête fréquente.
Sécurité	Rôle, permission et audit.

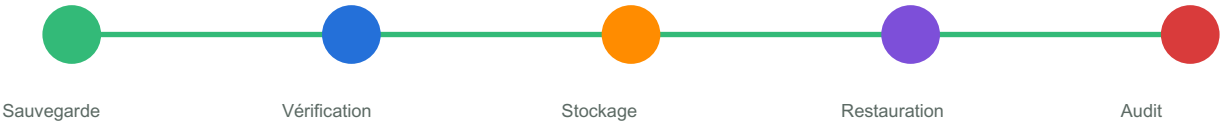
Gabarit de requête analytique

```
SELECT agence_id, COUNT(*) AS colis_enregistres
FROM colis
WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY agence_id
ORDER BY colis_enregistres DESC;
```

Fiche pratique base de données 168

Cette fiche sert de révision guidée. L'apprenant doit choisir un besoin métier, proposer un modèle, écrire une requête, prévoir un index, appliquer une règle de sécurité et définir une stratégie de sauvegarde.

Chaîne sauvegarde et restauration



Étape	Résultat attendu
Modèle	Tables, collections, relations ou documents.
Requête	Résultat exact, filtré et performant.
Index	Justification selon la requête fréquente.
Sécurité	Rôle, permission et audit.

Gabarit de requête analytique

```
SELECT agence_id, COUNT(*) AS colis_enregistres
FROM colis
WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY agence_id
ORDER BY colis_enregistres DESC;
```


Fiche pratique base de données 169

Cette fiche sert de révision guidée. L'apprenant doit choisir un besoin métier, proposer un modèle, écrire une requête, prévoir un index, appliquer une règle de sécurité et définir une stratégie de sauvegarde.

Maturité base de données



SQL



Index



Sécurité



Backup



NoSQL

Étape	Résultat attendu
Modèle	Tables, collections, relations ou documents.
Requête	Résultat exact, filtré et performant.
Index	Justification selon la requête fréquente.
Sécurité	Rôle, permission et audit.

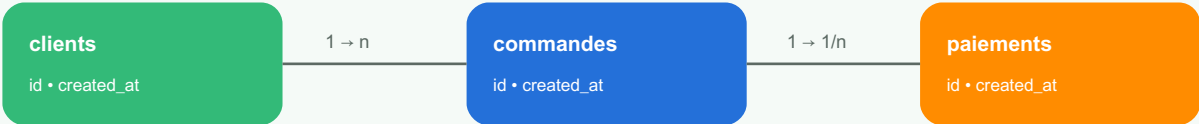
Gabarit de requête analytique

```
SELECT agence_id, COUNT(*) AS colis_enregistres
FROM colis
WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY agence_id
ORDER BY colis_enregistres DESC;
```

Fiche pratique base de données 170

Cette fiche sert de révision guidée. L'apprenant doit choisir un besoin métier, proposer un modèle, écrire une requête, prévoir un index, appliquer une règle de sécurité et définir une stratégie de sauvegarde.

Schéma relationnel simplifié



Étape	Résultat attendu
Modèle	Tables, collections, relations ou documents.
Requête	Résultat exact, filtré et performant.
Index	Justification selon la requête fréquente.
Sécurité	Rôle, permission et audit.

Gabarit de requête analytique

```
SELECT agence_id, COUNT(*) AS colis_enregistres
FROM colis
WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY agence_id
ORDER BY colis_enregistres DESC;
```

Fiche pratique base de données 171

Cette fiche sert de révision guidée. L'apprenant doit choisir un besoin métier, proposer un modèle, écrire une requête, prévoir un index, appliquer une règle de sécurité et définir une stratégie de sauvegarde.

Cycle de vie des données



Étape	Résultat attendu
Modèle	Tables, collections, relations ou documents.
Requête	Résultat exact, filtré et performant.
Index	Justification selon la requête fréquente.
Sécurité	Rôle, permission et audit.

Gabarit de requête analytique

```
SELECT agence_id, COUNT(*) AS colis_enregistres
FROM colis
WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY agence_id
ORDER BY colis_enregistres DESC;
```

Fiche pratique base de données 172

Cette fiche sert de révision guidée. L'apprenant doit choisir un besoin métier, proposer un modèle, écrire une requête, prévoir un index, appliquer une règle de sécurité et définir une stratégie de sauvegarde.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

Étape	Résultat attendu
Modèle	Tables, collections, relations ou documents.
Requête	Résultat exact, filtré et performant.
Index	Justification selon la requête fréquente.
Sécurité	Rôle, permission et audit.

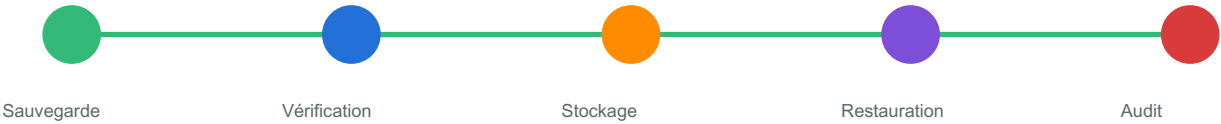
Gabarit de requête analytique

```
SELECT agence_id, COUNT(*) AS colis_enregistres
FROM colis
WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY agence_id
ORDER BY colis_enregistres DESC;
```

Fiche pratique base de données 173

Cette fiche sert de révision guidée. L'apprenant doit choisir un besoin métier, proposer un modèle, écrire une requête, prévoir un index, appliquer une règle de sécurité et définir une stratégie de sauvegarde.

Chaîne sauvegarde et restauration



Étape	Résultat attendu
Modèle	Tables, collections, relations ou documents.
Requête	Résultat exact, filtré et performant.
Index	Justification selon la requête fréquente.
Sécurité	Rôle, permission et audit.

Gabarit de requête analytique

```
SELECT agence_id, COUNT(*) AS colis_enregistres
FROM colis
WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY agence_id
ORDER BY colis_enregistres DESC;
```

Fiche pratique base de données 174

Cette fiche sert de révision guidée. L'apprenant doit choisir un besoin métier, proposer un modèle, écrire une requête, prévoir un index, appliquer une règle de sécurité et définir une stratégie de sauvegarde.

Maturité base de données



SQL



Index



Sécurité



Backup



NoSQL

Étape	Résultat attendu
Modèle	Tables, collections, relations ou documents.
Requête	Résultat exact, filtré et performant.
Index	Justification selon la requête fréquente.
Sécurité	Rôle, permission et audit.

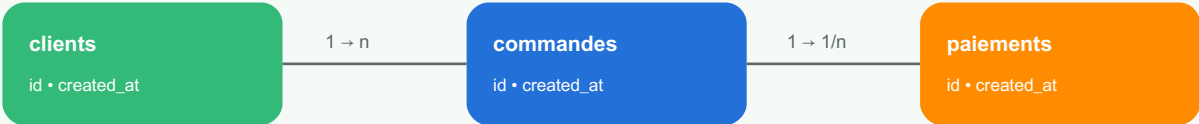
Gabarit de requête analytique

```
SELECT agence_id, COUNT(*) AS colis_enregistres
FROM colis
WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY agence_id
ORDER BY colis_enregistres DESC;
```

Fiche pratique base de données 175

Cette fiche sert de révision guidée. L'apprenant doit choisir un besoin métier, proposer un modèle, écrire une requête, prévoir un index, appliquer une règle de sécurité et définir une stratégie de sauvegarde.

Schéma relationnel simplifié



Étape	Résultat attendu
Modèle	Tables, collections, relations ou documents.
Requête	Résultat exact, filtré et performant.
Index	Justification selon la requête fréquente.
Sécurité	Rôle, permission et audit.

Gabarit de requête analytique

```
SELECT agence_id, COUNT(*) AS colis_enregistres
FROM colis
WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY agence_id
ORDER BY colis_enregistres DESC;
```

Fiche pratique base de données 176

Cette fiche sert de révision guidée. L'apprenant doit choisir un besoin métier, proposer un modèle, écrire une requête, prévoir un index, appliquer une règle de sécurité et définir une stratégie de sauvegarde.

Cycle de vie des données



Étape	Résultat attendu
Modèle	Tables, collections, relations ou documents.
Requête	Résultat exact, filtré et performant.
Index	Justification selon la requête fréquente.
Sécurité	Rôle, permission et audit.

Gabarit de requête analytique

```
SELECT agence_id, COUNT(*) AS colis_enregistres
FROM colis
WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY agence_id
ORDER BY colis_enregistres DESC;
```


Fiche pratique base de données 177

Cette fiche sert de révision guidée. L'apprenant doit choisir un besoin métier, proposer un modèle, écrire une requête, prévoir un index, appliquer une règle de sécurité et définir une stratégie de sauvegarde.

Structure logique d'un index



Index B-tree simplifié : réduire le coût de recherche.

Étape	Résultat attendu
Modèle	Tables, collections, relations ou documents.
Requête	Résultat exact, filtré et performant.
Index	Justification selon la requête fréquente.
Sécurité	Rôle, permission et audit.

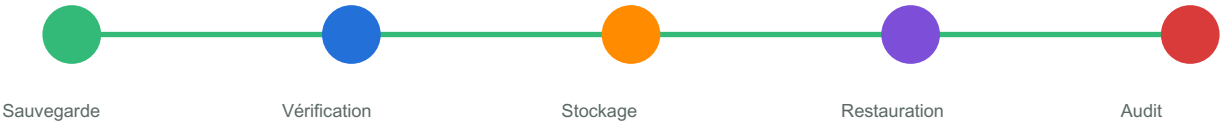
Gabarit de requête analytique

```
SELECT agence_id, COUNT(*) AS colis_enregistres
FROM colis
WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY agence_id
ORDER BY colis_enregistres DESC;
```

Fiche pratique base de données 178

Cette fiche sert de révision guidée. L'apprenant doit choisir un besoin métier, proposer un modèle, écrire une requête, prévoir un index, appliquer une règle de sécurité et définir une stratégie de sauvegarde.

Chaîne sauvegarde et restauration



Étape	Résultat attendu
Modèle	Tables, collections, relations ou documents.
Requête	Résultat exact, filtré et performant.
Index	Justification selon la requête fréquente.
Sécurité	Rôle, permission et audit.

Gabarit de requête analytique

```
SELECT agence_id, COUNT(*) AS colis_enregistres
FROM colis
WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY agence_id
ORDER BY colis_enregistres DESC;
```

Fiche pratique base de données 179

Cette fiche sert de révision guidée. L'apprenant doit choisir un besoin métier, proposer un modèle, écrire une requête, prévoir un index, appliquer une règle de sécurité et définir une stratégie de sauvegarde.

Maturité base de données



SQL



Index



Sécurité



Backup



NoSQL

Étape	Résultat attendu
Modèle	Tables, collections, relations ou documents.
Requête	Résultat exact, filtré et performant.
Index	Justification selon la requête fréquente.
Sécurité	Rôle, permission et audit.

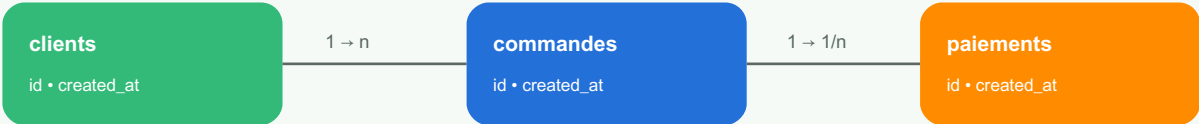
Gabarit de requête analytique

```
SELECT agence_id, COUNT(*) AS colis_enregistres
FROM colis
WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY agence_id
ORDER BY colis_enregistres DESC;
```

Fiche pratique base de données 180

Cette fiche sert de révision guidée. L'apprenant doit choisir un besoin métier, proposer un modèle, écrire une requête, prévoir un index, appliquer une règle de sécurité et définir une stratégie de sauvegarde.

Schéma relationnel simplifié



Étape	Résultat attendu
Modèle	Tables, collections, relations ou documents.
Requête	Résultat exact, filtré et performant.
Index	Justification selon la requête fréquente.
Sécurité	Rôle, permission et audit.

Gabarit de requête analytique

```
SELECT agence_id, COUNT(*) AS colis_enregistres
FROM colis
WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY agence_id
ORDER BY colis_enregistres DESC;
```